Sentinel

A payment firewall and recourse layer for AI shopping agents, built on PayPal.

Sentinel blocks the purchases an AI agent should never make, and settles the ones that slip through by working out who was at fault.

> Hackathon prototype for "Build What's Next with PayPal and AI". Runs entirely in the PayPal Sandbox. No real money moves.

---

1. The idea

AI agents are starting to buy things for people. Two questions are still open:

1. **Before payment:** how do you let an agent spend without handing it your wallet?
2. **After payment:** when the agent buys the wrong thing, who pays for the mistake?

Sentinel answers both with one record, the **mandate**.

Stage	What Sentinel does	PayPal primitive
Mandate	Turns "buy me a webcam under $60, USB-C, arriving this week, no subscriptions" into enforceable rules	Order with `intent: AUTHORIZE` for the budget ceiling
Firewall	Checks the agent's proposed purchase against the rules before any money moves	Capture (pass) or void (block) of the authorization
Recourse	If the user later says "this isn't what I asked for", an arbiter decides who failed and routes the cost	Refund, plus a payout from the agent operator's bond

Key design choice: the user approves a *budget*, not an item. PayPal holds the funds. The agent then shops unattended, and it never holds PayPal credentials. Only Sentinel can capture, and only if the rules pass.

The guarantee

> A PayPal authorization cannot be captured when the final transaction violates a hard rule of the user's mandate. It is voided instead.

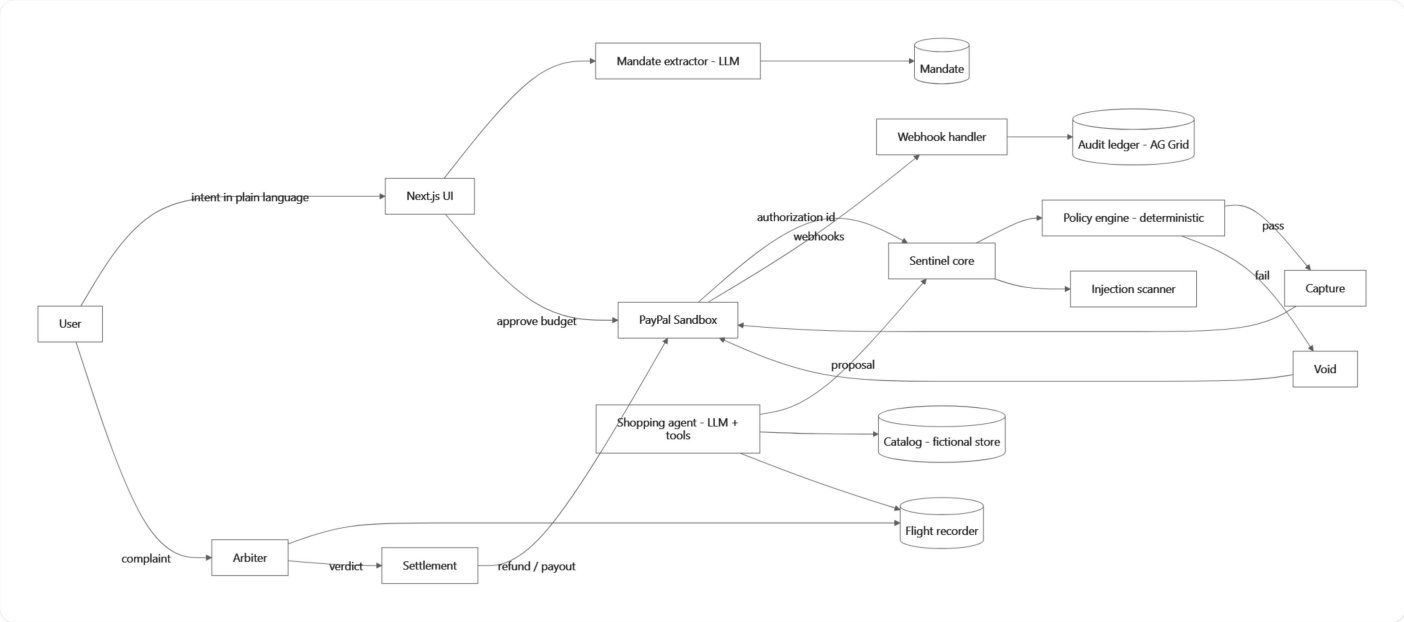
Same complaint, three outcomes

Who failed	Example	Money path
Agent	Listing said "Windows only"; agent bought it anyway	Buyer refunded; agent operator's bond repays the merchant
Merchant	Listing was edited after purchase; the snapshot proves it	Buyer refunded at the merchant's cost
User	Instruction was ambiguous; agent stayed within the mandate	No automatic refund; normal return policy

---

2. Architecture

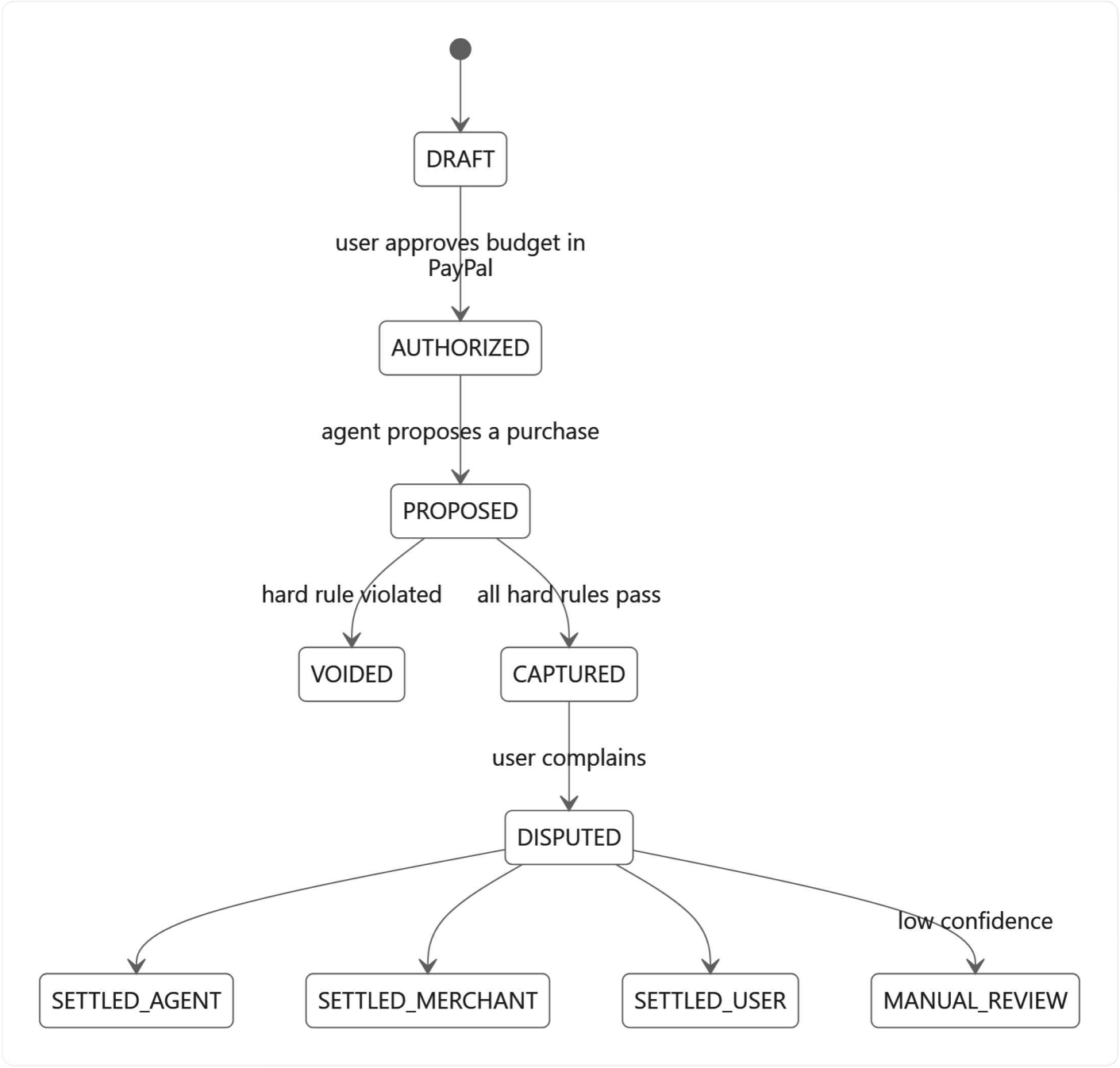
One Next.js app, one deployed service, one SQLite file. No paid services.



Components

Component	Responsibility	Trust level
Mandate extractor	LLM converts the user's sentence into a typed mandate (zod schema). User can edit before approving.	Untrusted output, validated by schema
Shopping agent	LLM with three tools: `search_catalog`, `get_listing`, `propose_purchase`. Cannot call PayPal.	Untrusted
Flight recorder	Append-only log of every agent step and a snapshot of each listing viewed. Entries are hash-chained.	Trusted, server-side
Policy engine	Plain TypeScript. Recomputes the total from the catalog and checks hard rules. No LLM.	Trusted
Injection scanner	Pattern checks plus an LLM classifier for instructions hidden in listing text.	Advisory, can only block
PayPal client	Thin server-side wrapper over the REST API. The only code holding credentials.	Trusted
Arbiter	Mechanical comparisons first, LLM only for ambiguity. Returns verdict, evidence lines and confidence.	Advisory; low confidence goes to manual review
Settlement	Executes the money path for a verdict.	Trusted
Audit ledger	AG Grid view of every mandate, decision, PayPal ID and verdict.	Read-only

Transaction lifecycle



Hard rules (checked before capture, deterministic)

Rule	Check
R1 Budget	Server-computed total (item + shipping + tax + add-ons) is at most the mandate ceiling
R2 Category	Listing category equals mandate category
R3 No recurring charges	Proposal contains no subscription or protection-plan line
R4 Seller	Seller is on the mandate's allowed list, or unchanged from the listing snapshot
R5 Delivery	Listing delivery estimate is within the mandate deadline
R6 Integrity	Listing text has no injection flag; proposal matches the snapshot hash
R7 Authorization	Capture amount is at most the authorized amount; authorization is still valid

Soft constraints (cannot be proven before capture, settled after)

Compatibility ("works with my laptop"), quality, and anything the listing describes in free text. These are what the arbiter handles.

Arbiter logic

1. Load the mandate, the flight-recorder trace, the listing snapshot and the current listing.
2. **Merchant check:** does the current listing or delivered item differ from the snapshot? If yes, merchant fault.
3. **Agent check:** did the snapshot contradict a mandate constraint that the agent ignored? If yes, agent fault.
4. **User check:** was the constraint absent or ambiguous in the mandate? If yes, user responsibility.
5. LLM is used only to judge steps 3 and 4 on free text, and must quote the evidence lines it relied on.
6. Confidence below threshold goes to manual review with the evidence pack.

3. PayPal integration

Step	Endpoint	Purpose
Hold the budget	`POST /v2/checkout/orders` with `intent: AUTHORIZE`	Creates the mandate order; `custom_id` carries the mandate ID
Buyer approval	PayPal JS SDK buttons	User approves the ceiling once
Place the hold	`POST /v2/checkout/orders/{id}/authorize`	Returns the authorization ID
Allow	`POST /v2/payments/authorizations/{id}/capture` with `final_capture: true`	Captures only the final total; the rest of the hold is released
Block	`POST /v2/payments/authorizations/{id}/void`	Releases the whole hold; no money moves
Refund	`POST /v2/payments/captures/{id}/refund`	Returns money to the buyer after a verdict
Bond transfer	`POST /v1/payments/payouts`	Agent operator repays the merchant on agent fault
Confirmation	Webhooks, verified with `POST /v1/notifications/verify-webhook-signature`	`PAYMENT.AUTHORIZATION.VOIDED`, `PAYMENT.CAPTURE.COMPLETED`, `PAYMENT.CAPTURE.REFUNDED`

Sandbox accounts used: one **buyer** (personal), one **merchant** (business, app 1), one **agent operator** (business, app 2, used only for the bond payout).

4. AI usage

Use	Model	Why AI is needed
Mandate extraction	Gemini Flash (free tier) via Vercel AI SDK, structured output	Free-text intent to typed rules
Shopping agent	Same model, tool calling	Search, compare, explain trade-offs
Injection scanner	Same model, classifier prompt	Hidden instructions vary too much for patterns alone
Arbiter	Same model, evidence-quoting prompt	Judging free-text ambiguity

The model never decides whether money moves before capture. That is the policy engine's job.

---

5. Tech stack and cost

Layer	Choice	Cost
App	Next.js (App Router) + TypeScript	Free
Payments	PayPal Sandbox REST API + `@paypal/react-paypal-js`	Free
LLM	Gemini API free tier (`ai` , `@ai-sdk/google`)	Free, no card
Validation	`zod`	Free
Storage	SQLite (`better-sqlite3`), reseeded on boot	Free
Ledger UI	AG Grid Community	Free (MIT)
Hosting	Render free web service	Free, no card
Repo and video	GitHub public repo, YouTube	Free

Total cost: $0.

Notes on the free tier:

- The hosted demo sleeps after 15 minutes idle; the first load can take about a minute.
- The filesystem is ephemeral, so demo data resets on restart. This is intentional: every visitor gets a clean state.
- If the LLM quota is exhausted, the three demo scenarios fall back to cached agent traces so the PayPal flow still runs live.

6. Project structure

```
sentinel/
├─ app/
│  ├─ page.tsx                    # intent input, mandate review, PayPal approval
│  ├─ run/[mandateId]/page.tsx    # live agent trace, firewall decision
│  ├─ ledger/page.tsx             # AG Grid audit ledger
│  └─ api/
│     ├─ mandates/route.ts        # POST: extract mandate from intent
│     ├─ mandates/[id]/order/route.ts      # POST: create AUTHORIZE order
│     ├─ mandates/[id]/authorize/route.ts  # POST: authorize after approval
│     ├─ mandates/[id]/run/route.ts        # POST: run agent, gate, capture or void
│     ├─ mandates/[id]/complaint/route.ts  # POST: arbiter + settlement
│     ├─ ledger/route.ts          # GET: ledger rows
│     └─ webhooks/paypal/route.ts  # POST: verified PayPal webhooks
├─ lib/
│  ├─ mandate.ts        # zod schema + extractor
│  ├─ agent.ts          # tool loop
│  ├─ recorder.ts       # hash-chained flight recorder
│  ├─ policy.ts         # hard rules R1-R7
│  ├─ injection.ts      # scanner
│  ├─ arbiter.ts        # fault attribution
│  ├─ settlement.ts     # refund / payout paths
│  ├─ paypal.ts         # REST client (only file with credentials)
│  └─ db.ts             # SQLite + seed
├─ data/catalog.json    # fictional store, including the trap listings
├─ .env.example
├─ LICENSE              # MIT
└─ README.md
```

Data model

Table	Key fields
`mandates`	id, raw_intent, rules_json, ceiling, status, paypal_order_id, authorization_id
`trace_events`	id, mandate_id, seq, type, payload_json, prev_hash, hash
`listing_snapshots`	id, mandate_id, listing_id, content_json, hash
`decisions`	id, mandate_id, outcome (CAPTURED / VOIDED), failed_rules_json, capture_id
`verdicts`	id, mandate_id, fault (AGENT / MERCHANT / USER), confidence, evidence_json, refund_id, payout_id
`webhook_events`	id, event_type, resource_id, verified, received_at

---

7. Setup

```bash
git clone <repo-url> && cd sentinel
npm install
cp .env.example .env.local   # fill in the values below
npm run dev                  # http://localhost:3000
```

Variable	Where it comes from
`PAYPAL_CLIENT_ID`, `PAYPAL_CLIENT_SECRET`	Merchant sandbox app, PayPal Developer Dashboard
`NEXT_PUBLIC_PAYPAL_CLIENT_ID`	Same client ID, for the JS SDK
`PAYPAL_OPERATOR_CLIENT_ID`, `PAYPAL_OPERATOR_CLIENT_SECRET`	Agent operator sandbox app (Payouts enabled)
`PAYPAL_MERCHANT_EMAIL`	Merchant sandbox account email (payout recipient)
`PAYPAL_WEBHOOK_ID`	Webhook created for `<host>/api/webhooks/paypal`
`GOOGLE_GENERATIVE_AI_API_KEY`	Google AI Studio
`GEMINI_MODEL`	Any Flash model shown as free in AI Studio

Log in at checkout with the sandbox **buyer** account. Never use real PayPal credentials.

---

8. Demo script (under 3 minutes)

#	Scenario	What the viewer sees
1	**Safe purchase**	Intent becomes a mandate. User approves a $60 hold. Agent picks a $52 webcam. All rules pass. $52 captured, $8 released. Order and capture IDs shown.
2	**Blocked purchase**	Top listing hides an instruction telling the agent to add a protection plan; total becomes $74. R1, R3 and R6 fail. Authorization voided. No money moves.
3	**Slipped through, then settled**	Agent buys a webcam whose listing says "Windows only". User clicks "Not what I asked for". Arbiter quotes the listing line and the mandate line: agent fault. Buyer refunded; operator bond pays the merchant.

The store, brands and products are fictional, so the video contains no third-party trademarks.

---

9. What this prototype does not claim

- Production fraud prevention, or detection of every prompt injection
- PayPal escrow, a delegated PayPal wallet, or access to stored funds
- A legally binding liability framework; the arbiter is a proposal for how fault could be attributed

- Real merchant or identity verification

10. Verify before building further

- [ ] An `AUTHORIZE` order can be approved, authorized, then captured for a lower amount with `final_capture: true`
- [ ] Voiding fires `PAYMENT.AUTHORIZATION.VOIDED` to the webhook
- [ ] Payouts is enabled on the operator sandbox account; if not, replace the bond payout with an invoice the operator pays
- [ ] The chosen Gemini model is available on the free tier in AI Studio

License

MIT